

TP3

IPCA

David Ferreira — a35324

24 Maio 2026

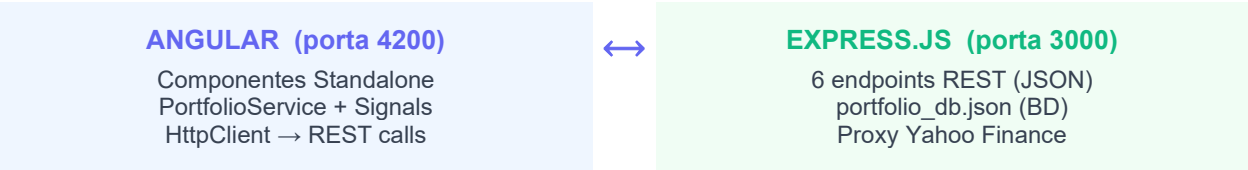
1. Visão Geral do Projeto

O Stock Portfolio Manager é uma aplicação web full-stack que permite gerir uma carteira de ações: registar compras, consultar a valorização em tempo real, visualizar gráficos de alocação e importar/exportar dados em JSON. Foi desenvolvido como TP3, com cotações de demonstração para 05/05/2026 (MSFT=\$330 e TSLA=\$224 conforme o guia).

Framework Frontend	Angular 21 com Standalone Components
Backend	Node.js + Express.js 4.x (porta 3000)
Base de Dados	Ficheiro JSON local (portfolio_db.json)
API de Cotações	Yahoo Finance v7 (via proxy Express)
Linguagem	TypeScript 5.9 com modo strict
Testes	Vitest 4.x

2. Arquitetura e Estrutura de Pastas

A aplicação divide-se em duas camadas independentes que comunicam via HTTP/REST:



Estrutura de pastas principal:

```
src/app/components/ → 7 componentes standalone (dashboard, navbar, charts, table, modal, cards, import-export)
src/app/services/ → PortfolioService (estado global com Signals)
src/app/app.config.ts → providers: provideRouter, provideHttpClient
server/server.js → REST API Express com 6 endpoints
server/portfolio_db.json → base de dados JSON persistente
```

3. Componentes Angular

Todos os componentes são standalone (sem NgModule) e injetam o PortfolioService com a nova API inject():

Componente	Responsabilidade
AppComponent	Raiz. Compõe NavbarComponent + DashboardComponent
NavbarComponent	Status REST, toggle Demo/Live, tema dark/light, perfil do utilizador
DashboardComponent	Layout principal em bento-grid. Orquestra todos os sub-componentes
SummaryCardsComponent	4 KPI cards: Valor Aquisição, Valor Atual, Lucro/Prejuízo, N.º Ativos
PortfolioChartsComponent	Donut chart SVG de alocação + barras de comparação custo vs valor
PortfolioTableComponent	Tabela de transações com variações coloridas e linha de totais
TransactionModalComponent	Modal slide-in para adicionar nova posição com formulário validado

ImportExportComponent

Drag-and-drop para importar JSON + exportação do portfólio completo

4. PortfolioService — Gestão de Estado com Signals

O PortfolioService centraliza todo o estado da aplicação usando Angular Signals, a nova API reativa do Angular 16+ (estabilizada no 17). Signals substituem BehaviorSubjects e NgRx para estado local: quando um sinal muda, apenas os consumidores desse sinal re-renderizam.

Signal / Computed	Descrição e Tipo
portfolioItems	signal<Transaction[]> — lista reativa de todas as transações em carteira
stockQuotes	signal<Record<string, StockQuote>> — mapa ticker → cotação atual
priceMode	signal<'demo' 'live'> — modo de preços ativo
isLoading / isBackendConnected	signal<boolean> — estados de carregamento e conectividade REST
totalInvested	computed() — soma de (qty × purchasePrice) para todas as posições
totalCurrentValue	computed() — soma de (qty × currentPrice) com cotações em tempo real
totalGainLoss / %	computed() — P&L absoluto e percentual da carteira completa

O serviço usa um effect() para refetch automático de cotações sempre que o portfólio ou o modo muda:

```
effect(() => {
  const tickers = this.portfolioItems().map(i => i.ticker); // lê o signal
  if ([...new Set(tickers)].length > 0) this.fetchQuotes(tickers); // reage
});
```

Todas as chamadas HTTP usam firstValueFrom() para converter Observables em Promises, permitindo async/await:

```
async loadPortfolio(): Promise<void> {
  const items = await firstValueFrom(this.http.get<Transaction[]>('/api/portfolio'));
  this.portfolioItems.set(items || []); // signal atualizado → UI re-renderiza
}
```

5. REST API — Express.js

O servidor Express (server/server.js) corre na porta 3000 com middleware cors() e express.json(). Serve como proxy para o Yahoo Finance (eliminando CORS no browser) e persiste dados em portfolio_db.json.

Método	Endpoint	Descrição	Parâmetros / Body
GET	/api/portfolio	Lista todas as transações	—
POST	/api/portfolio	Adiciona nova transação	{ ticker, company, quantity, purchasePrice, purchaseDate }
DELETE	/api/portfolio/:id	Remove transação por ID	Parâmetro de rota: id
GET	/api/stocks/quotes	Cotações de um ou mais tickers	?symbols=MSFT,TSLA&mode=demo live
POST	/api/portfolio/import	Substitui BD com array importado	Array de transacoes JSON
GET	/api/portfolio/export	Download do portfolio_db.json	—

Lógica de cotações — hierarquia de fallback no endpoint /api/stocks/quotes:

1. Demo: valores fixos acadêmicos (MSFT=\$330, TSLA=\$224 para 05/05/2026)

2. **Live:** pedido real a `query1.finance.yahoo.com/v7/finance/quote`

3. **Fallback:** valores mock para AAPL/AMZN/NVDA ou \$150 para qualquer ticker desconhecido

6. Como o Angular Funciona Neste Projeto

Bootstrap e Configuração

O Angular 21 usa a nova API standalone sem AppModule. O main.ts faz bootstrapApplication(App, appConfig) onde appConfig define os providers globais: provideRouter(routes), provideHttpClient() e provideBrowserGlobalErrorListeners().

Formulários e Validacao

O TransactionModalComponent usa Template-Driven Forms (FormsModule) com two-way binding [(ngModel)] e validacao de template (required, min). O botao de submit fica desativado enquanto txForm.invalid for verdadeiro. Inclui autocomplete de empresas por ticker (dicionario interno com AAPL, MSFT, TSLA, NVDA, etc.).

Routing

O ficheiro app.routes.ts esta vazio — toda a aplicacao corre numa unica rota raiz. O routing com multiplas paginas (ex.: /portfolio, /analytics) esta planeado mas nao implementado.

Sistema de Design — CSS Variables

Em vez de Bootstrap ou Tailwind, o projeto usa um design system proprio em styles.css com tokens CSS:

- Dois temas: :root (dark, inspirado em Vercel/Linear) e [data-theme=light]
- Variaveis de cor: --accent-primary (#6366F1), --color-green, --color-red, --text-primary
- Geometria: --radius-xs a --radius-lg, --shadow-sm a --shadow-lg
- Glassmorphism: --glass-blur: blur(12px) nos cards
- Fontes: Inter (corpo) + Outfit (titulos) via Google Fonts

7. Configuracoes do Projeto

angular.json — Builder	@angular/build:application (Vite/ESBuild, mais rapido que Webpack)
angular.json — Budget	Aviso 500kB / Erro 1MB no bundle inicial de producao
tsconfig.json — Modo	strict: true, noImplicitReturns, strictTemplates (Angular)
tsconfig.json — Target	ES2022 com module: preserve para tree-shaking
npm run dev	concurrently arranca Express (3000) + ng serve (4200) em simultaneo
npm run build	ng build — compila para dist/ com otimizacoes de producao
Formatacao	Prettier 3.8 + .editorconfig (indent=2, utf-8)
Testes	Vitest 4.x (substitui Karma/Jasmine no Angular 21)

8. Estado Atual — Implementado vs Incompleto

Implementado

- 6 endpoints REST completos com validacao e tratamento de erros
- Persistencia em JSON com leitura/escrita atomica no servidor
- 7 componentes Angular standalone com gestao de estado via Signals
- Cotacoes em modo Demo (fixo) e Live (Yahoo Finance em tempo real)

- Graficos: donut chart SVG calculado matematicamente + barras de comparacao
- Tabela com variacoes coloridas, formatacao europeia e linha de totais
- Importacao via drag-and-drop e exportacao de JSON
- Badge de status de conectividade REST na navbar
- Tema dark/light comutavel sem reload

Incompleto / Melhorias Possiveis

- Routing: app.routes.ts vazio — sem navegacao entre paginas
- Testes unitarios: ficheiros .spec.ts existem mas sem implementacao
- Edicao de transacoes: nao existe endpoint PUT nem UI de edicao
- Sem autenticacao — qualquer utilizador acede a carteira
- BD em ficheiro JSON nao escala para producao (sem controlo de concorrencia)

9. Conclusao

O Stock Portfolio Manager demonstra uma integracao solida entre Angular 21 e Express.js. Os pontos tecnicamente mais relevantes sao o uso de Angular Signals (substituindo NgRx/BehaviorSubjects para estado local), a arquitetura de proxy REST que resolve CORS e protege a logica de cotacoes, e o design system proprio com dois temas sem dependencias externas de CSS.

O projeto cumpre os requisitos academicos do TP3, incluindo os valores de demonstracao exatos para 05/05/2026, a formatacao europeia das variacoes percentuais (3,1% e 1,81%) e a estrutura de tabela exigida no guia de avaliacao.

Como proximos passos, seria prioritario completar o routing Angular, implementar testes com Vitest e substituir o ficheiro JSON por uma base de dados adequada para producao.

10. Link Github

<https://github.com/a35324-David/TP3.git>
